

Assignment 1: Hybrid Recommender System

Students per Group	Total Marks	Deadline
2 to 3 students	10 marks	23 April 2026

1. Objective

In this assignment you will design and implement a **Hybrid Recommender System** that combines **Association Rule Mining** with a **second complementary technique** of your choice. The goal is to understand the strengths and weaknesses of each method individually, and to demonstrate how combining them produces better recommendations than either method alone.

2. Background & Core Concepts

2.1 Association Rule Mining

Association rule mining discovers relationships between items in large transactional datasets. Given a set of transactions (e.g., movies watched, products purchased), the algorithm finds rules of the form:

```
{Item_A, Item_B} → {Item_C}
```

Read as: "Users who interacted with A and B also interacted with C"

Key metrics you must understand and report:

Metric	Formula	Meaning
Support	$\text{support}(A \rightarrow B) = \text{count}(A \cup B) / N$	How often A and B appear together across all transactions
Confidence	$\text{conf}(A \rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$	Given that A was seen, how likely is B? (conditional probability)
Lift	$\text{lift}(A \rightarrow B) = \text{conf}(A \rightarrow B) / \text{support}(B)$	Is the association stronger than random chance? Lift > 1 means yes

Algorithms: Use **Apriori** or **FP-Growth**. FP-Growth is significantly faster on large datasets because it avoids repeated dataset scans. Both are available in Python's `mlxtend` library.

2.2 TF-IDF for Content-Based Similarity

TF-IDF (Term Frequency – Inverse Document Frequency) converts item descriptions (movie plots, product descriptions, song tags) into numerical vectors. These vectors can then be compared using **cosine similarity** to find items with similar content — independently of what other users have done.

Step-by-step: How a movie TF-IDF vector is built

Step 1 – Build the document corpus. Treat each movie's metadata as a single text document. Combine title, genre, director, cast, and plot keywords into one string:

Movie	Combined Text Document
Inception	"sci-fi thriller dream subconscious heist Nolan DiCaprio time layers mind"
Interstellar	"sci-fi drama space wormhole time gravity Nolan Cooper NASA survival"
The Dark Knight	"action crime batman joker Gotham villain vigilante Nolan Bale"

Step 2 – Compute Term Frequency (TF). For each term t in movie document d :

$$TF(t, d) = \frac{\text{(number of times } t \text{ appears in } d)}{\text{(total number of terms in } d)}$$

Example: if "Nolan" appears 2 times in Inception (20 total terms)
 $TF(\text{"Nolan"}, \text{Inception}) = 2 / 20 = 0.10$

Step 3 – Compute Inverse Document Frequency (IDF). IDF penalises terms that appear in many documents (they are less informative):

$$IDF(t) = \log(N / df(t))$$

N = total number of movies in the corpus

$df(t)$ = number of movies containing term t

Example (corpus = 3 movies):

"Nolan" appears in all 3 movies → $IDF = \log(3/3) = 0.00$

"dream" appears in 1 movie → $IDF = \log(3/1) = 1.10$

"space" appears in 2 movies → $IDF = \log(3/2) = 0.41$

Step 4 – Multiply to get TF-IDF weight. The final weight of term t in movie d is:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

"dream" in Inception: $TF=0.05$, $IDF=1.10$ → $TF-IDF = 0.055$

"Nolan" in Inception: $TF=0.10$, $IDF=0.00$ → $TF-IDF = 0.000$

Notice: common words like "Nolan" get weight 0 (not discriminative)

Rare, specific words like "dream" get high weight (very informative)

Step 5 – Build the vector and compute cosine similarity. Each movie is now a vector of TF-IDF weights, one per unique term.

```
Inception      = [dream:0.055, space:0, subconscious:0.070, Nolan:0, ...]
Interstellar   = [dream:0,      space:0.041, gravity:0.065, Nolan:0, ...]

cosine_similarity(Inception, Interstellar)
               A · B
=  _____
      ||A|| × ||B||

Result: 0 (No similarity)
```

In Python this entire process is one line using `TfidfVectorizer` from `scikit-learn`, followed by `cosine_similarity`. The power is that you never need to manually set thresholds – every pair of items gets a continuous similarity score between 0 and 1.

3. Task Description

3.1 Choose Your Application Domain

Select **one** of the following domains or **suggest another domain**:

#	Domain	Example Dataset	Item Content Features
1	Movies (Netflix-like)	MovieLens 1M / MovieLens 100K (grouplens.org)	Genre, director, cast, plot keywords, year
2	E-commerce (Amazon-like)	Amazon Product Reviews (Kaggle), Online Retail UCI	Category, description, brand, price range
3	Music (Spotify-like)	Last.fm dataset, Spotify API tracks	Genre, tempo, energy, danceability, artist tags
4	News Articles	MIND dataset (Microsoft News)	Category, headline, abstract, named entities
5	Deduce Drug to Drug Interaction	FAERS — FDA Adverse Event Reporting System Each report = one "transaction" containing all drugs a patient was taking + the adverse events they experienced. You mine rules like: <i>{Warfarin, Aspirin} → {Bleeding}</i> DrugBank Best for content-based / hybrid component. A curated database of drug properties, mechanisms, targets, and known interactions. Each drug has a rich text description — perfect for the TF-IDF side of your hybrid system.	You mine either co-prescription patterns (what drugs appear together) or adverse event patterns (what drug combinations lead to reported side effects).

Cairo University – Faculty of Computers and Artificial Intelligence SIS317 Data Mining Spring 2026 Dr. Shaimaa Galal			Assignment 1 • 10 Marks
6	Other Medical Domain	Symptom → Disease diagnosis Mine rules like {fever, cough, fatigue} → {influenza}. Lab results → Diagnosis {high_glucose, high_HbA1c, high_BMI} → {diabetes}. Dataset: UCI Pima Indians Diabetes, or MIMIC lab events table.	

3.2 Choose Your Hybrid Approach

You must combine **Association Rule Mining** with any second technique (you can propose another one). Sample Techniques are provided in the following table. **Justify your choice in your report.**

Option	Second Technique	How It Combines with AR	Difficulty
A	TF-IDF Cosine Similarity	AR confidence + TF-IDF cosine similarity, weighted by α	☆☆ Intermediate
B	User-Based Collaborative Filtering (cosine on user vectors)	AR finds item correlations; CF finds similar users. Average their top-k lists	☆☆ Intermediate

3.3 The Hybrid Scoring Formula (for Option A — generalise as needed)

Weighted Hybrid Score

```

hybrid_score(candidate) =  $\alpha$  × AR_score(candidate)
                        + (1 -  $\alpha$ ) × content_score(candidate)

AR_score      = confidence of the best matching rule (or lift,
normalised)
content_score = TF-IDF cosine similarity between candidate and watched
items
 $\alpha \in [0, 1]$  = tunable weight (you must experiment with at least 3
values)

 $\alpha = 1.0$  → pure association rules
 $\alpha = 0.0$  → pure content-based
 $\alpha = 0.5$  → equal hybrid (start here)

```

Adapt this formula to your chosen hybrid option. For Option B, replace content_score with the CF similarity score.

4. Input and Output Specification

4.1 System Input

- **Dataset:** user–item interaction file (CSV or similar) plus item metadata file.
- **AR parameters:** minimum support threshold, minimum confidence threshold, minimum lift threshold.
- **Query:** a user ID (or a seed set of item IDs for cold-start testing).
- **Hybrid weight:** the value of α .
- **k:** number of recommendations to return.

4.2 System Output

For each query user, your system must print, then **recommend movies to watch**:

Example output for a movie system (User ID: 42, $\alpha=0.5$, $k=5$):

```
Seed movies (watched): The Matrix, Inception

--- Association Rules Candidates ---
Interstellar      conf=0.67  lift=1.80
The Dark Knight   conf=0.33  lift=1.20

--- TF-IDF Content Candidates ---
Interstellar      cosine=0.72
Gravity           cosine=0.58

--- Hybrid Top-5 Recommendations ( $\alpha=0.5$ ) ---
1. Interstellar    hybrid_score=0.695
2. The Dark Knight hybrid_score=0.470
...
```

Note: Evaluate Your System

Evaluation is **mandatory**. You must hold out a portion of each user's interactions as the **ground truth** and measure whether your system can predict them. **Ground truth** can be obtained from another dataset or annotated manually if it is hard to find a dataset online.

Evaluation is done by computing Precision and Recall for each user recommendation. Also, you can suggest other metrics but explain why.

5. Help Notes & Recommended Libraries

Task	Library / Tool	Key Function
Association Rules	Python: mlxtend	<code>apriori()</code> , <code>association_rules()</code>
Association Rules (fast)	Python: efficient-apriori	<code>apriori()</code>
TF-IDF Vectors	Python: scikit-learn	<code>TfidfVectorizer</code> , <code>cosine_similarity</code>
User-CF	Python: scikit-learn	<code>NearestNeighbors</code> or manual cosine
Evaluation (Precision/Recall)	Python: scikit-learn	custom loop or <code>ndcg_score</code>

8. Submission Requirements

1. **Code:** Well-commented implementation in Python (preferred) or any other language in a zipped folder.
2. **Report (word document or PDF):** Must include:
 - Domain and dataset description (why you chose it, how you preprocessed it, source).
 - Chosen hybrid option and justification.
 - AR parameters (min_support, min_confidence, min_lift) and how you chose them.
 - TF-IDF (or other) construction.
 - Hybrid scoring formula and values of α tested.
 - Evaluation table (Precision and Recall).
 - Results and Discussion: which method works best and why?
 - Is using a hybrid approach beneficial in your tested domain? Or it has been enough to use AR or content-based only? Prove by results.
3. **Screenshots / Output Logs:** Show at least 3 sample recommendation outputs (different users or seed items).
4. **Bonus (optional):** Complexity of the field adds 1 bonus mark.